

Redux Thunk

Redux Thunk est un middleware utilisé avec Redux pour gérer les actions asynchrones comme les appels API (fetch, axios). Par défaut, Redux ne gère que des actions synchrones (des objets simples). Redux Thunk permet à une action de retourner une fonction.

Dans une application réelle, on a souvent besoin de :

- Charger des données depuis une API
- Gérer l'état de chargement (loading)
- Gérer les erreurs

Redux seul ne sait pas gérer ces opérations asynchrones. Redux Thunk résout ce problème.

Principe de fonctionnement

Une action Redux classique retourne un objet :
{ type: 'ACTION' }

Avec Redux Thunk, une action peut retourner une fonction.
Cette fonction reçoit automatiquement dispatch comme argument.

Exemple (Thunk classique)

Action Thunk

Cette fonction est appelée depuis le composant React. Elle retourne une fonction asynchrone. Redux Thunk injecte automatiquement dispatch.

```
export const fetchStagiaires = () => {  
  return async (dispatch) => {  
    // Indique que le chargement commence  
    dispatch({ type: 'FETCH_STAGIAIRES_START' });  
  
    try {  
      // Appel API  
      const response = await fetch('http://127.0.0.1:8000/api/stagiaires');  
      const data = await response.json();  
  
      // Succès : données reçues  
      dispatch({ type: 'FETCH_STAGIAIRES_SUCCESS', payload: data });  
    } catch (error) {  
      // Erreur lors de l'appel API  
      dispatch({ type: 'FETCH_STAGIAIRES_ERROR', payload: error.message });  
    }  
  }  
}
```

```
    }  
  };  
};
```

Reducer

Le reducer reçoit les actions envoyées par dispatch et met à jour le state.

```
const initialState = {  
  stagiaires: [],  
  loading: false,  
  error: null  
};  
  
function stagiaireReducer(state = initialState, action) {  
  switch (action.type) {  
    case 'FETCH_STAGIAIRES_START':  
      return { ...state, loading: true, error: null };  
  
    case 'FETCH_STAGIAIRES_SUCCESS':  
      return { ...state, loading: false, stagiaires: action.payload };  
  
    case 'FETCH_STAGIAIRES_ERROR':  
      return { ...state, loading: false, error: action.payload };  
  
    default:  
      return state;  
  }  
}
```

Store Redux

Redux Thunk doit être ajouté comme middleware pour permettre les actions asynchrones.

```
import { createStore, applyMiddleware } from 'redux';  
import thunk from 'redux-thunk';  
  
const store = createStore(stagiaireReducer, applyMiddleware(thunk));
```

Utilisation dans un composant React

useEffect permet de déclencher l'appel API une seule fois au premier rendu.

```
const dispatch = useDispatch();
```

```
useEffect(() => {
```

```
dispatch(fetchStagiaires());  
}, []);
```

Exemple AVEC Redux Toolkit

Redux Toolkit simplifie Redux et inclut Redux Thunk par défaut.

createAsyncThunk

createAsyncThunk génère automatiquement trois états :

- pending (chargement)
- fulfilled (succès)
- rejected (erreur)

```
export const fetchStagiaires = createAsyncThunk(  
  'stagiaires/fetchStagiaires',  
  async () => {  
    const response = await fetch('http://127.0.0.1:8000/api/stagiaires');  
    return response.json();  
  }  
);
```

createSlice

extraReducers permet de gérer les différents états de l'action asynchrone.

```
const stagiaireSlice = createSlice({  
  name: 'stagiaires',  
  initialState: { stagiaires: [], loading: false, error: null },  
  extraReducers: (builder) => {  
    builder  
      .addCase(fetchStagiaires.pending, (state) => {  
        state.loading = true;  
      })  
      .addCase(fetchStagiaires.fulfilled, (state, action) => {  
        state.loading = false;  
        state.stagiaires = action.payload;  
      })  
      .addCase(fetchStagiaires.rejected, (state, action) => {  
        state.loading = false;  
        state.error = action.error.message;  
      });  
  }  
});
```

5. Comparaison

Thunk classique : plus de code, mais très pédagogique.

Redux Toolkit : moins de code, plus moderne, recommandé en production.